

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Título del Práctica:

SmartBot

PONDERACIÓN: 10 pts

 **Tiempo estimado: 25 hrs/min**

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Habilidad(es) blandas a formar.....	3
2. Resultado del Aprendizaje.....	3
2.1. Objetivo SMART.....	3
3. Enunciado de la Práctica.....	4
3.1 Descripción del problema a resolver.....	4
3.2 Alcance de la práctica.....	4
Restricciones.....	5
Lenguajes y herramientas.....	6
Requisitos mínimos del sistema.....	6
4. Entregables.....	7
5. Material de apoyo.....	8
Documentación oficial.....	8
6. Recursos y herramientas a utilizar.....	8
Software / Hardware.....	8
Plataformas.....	9

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Innovación	El estudiante desarrolla una solución tecnológica que automatiza la atención de consultas mediante un bot de Telegram, integrando diferentes tecnologías para resolver necesidades de comunicación y acceso a la información.

1.2. Competencia(s)

Diseñar e implementar sistemas de atención automatizada mediante bots de Telegram, integrando APIs REST, bases de datos y herramientas de desarrollo backend para gestionar y responder consultas de manera eficiente.

1.3. Habilidad(es) blandas a formar

La práctica le permitirá desarrollar las siguientes habilidades:

- Pensamiento lógico y analítico.
- Resolución de problemas.
- Atención al detalle.
- Capacidad de investigación.
- Organización y estructuración de conocimiento.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Desarrollar un sistema de respuestas automatizadas mediante un bot de Telegram conectado a una API REST y una base de datos.	El sistema deberá permitir gestionar al menos 15 preguntas frecuentes, 15 respuestas asociadas y realizar operaciones completas de creación, consulta, actualización y eliminación de registros.	Implementando un backend en python, una base de datos relacional o documental, un bot de Telegram funcional y una interfaz administrativa para la	Para fortalecer las competencias en integración de sistemas, desarrollo de APIs, automatización de consultas y construcción de aplicaciones orientadas a servicios.	Al finalizar la práctica y entregar el proyecto en la fecha establecida.

		gestión de contenido.		
--	--	-----------------------	--	--

3. Enunciado de la Práctica

3.1 Descripción del problema a resolver

En diversos entornos académicos, empresariales y de soporte técnico, es común que los usuarios realicen consultas repetitivas relacionadas con información previamente conocida, como horarios, procedimientos, requisitos, fechas importantes o preguntas frecuentes. Atender estas consultas de forma manual puede consumir tiempo y recursos, además de generar retrasos en la entrega de información.

Ante esta necesidad, se propone el desarrollo de **SmartBot**, un sistema de respuestas automatizadas basado en un bot de Telegram capaz de responder consultas utilizando información almacenada en una base de datos. El sistema deberá permitir que los usuarios interactúen con el bot mediante mensajes y comandos, obteniendo respuestas de manera inmediata y automatizada.

Adicionalmente, la solución deberá contar con un módulo administrativo accesible mediante una interfaz web, desde el cual se podrán gestionar las preguntas, respuestas y categorías disponibles en el sistema. Esto permitirá actualizar el conocimiento del bot sin necesidad de modificar el código fuente.

La práctica busca que el estudiante aplique conceptos de desarrollo backend, diseño de APIs REST, integración con servicios externos, gestión de bases de datos y automatización de procesos mediante bots conversacionales, construyendo una solución funcional orientada a la atención de consultas frecuentes.

3.2 Alcance de la práctica

La práctica deberá incluir como mínimo los siguientes componentes:

- Desarrollo de un bot funcional utilizando la plataforma Telegram.
- Implementación de un backend utilizando el lenguaje python.
- Desarrollo de una API REST para la comunicación entre el bot, la base de datos y el módulo administrativo.
- Implementación de una base de datos para el almacenamiento de preguntas, respuestas y categorías.
- Registro de al menos 20 preguntas frecuentes y sus respectivas respuestas.
- Implementación de operaciones CRUD (Crear, Consultar, Actualizar y Eliminar) para la gestión de preguntas y respuestas.
- Desarrollo de una interfaz web administrativa para la gestión de la información del bot.

- El sistema deberá implementar un mecanismo de autenticación para el acceso al panel administrativo. Para fines de evaluación, deberá existir un usuario preconfigurado con las siguientes credenciales:
 - Usuario: **IA1-User**
 - password: IA1-password@_new
- Consulta de información desde el bot mediante mensajes enviados por los usuarios.
- Respuesta automática del bot utilizando la información almacenada en la base de datos.
- Manejo de mensajes cuando no exista una respuesta registrada para la consulta realizada.
- Uso de control de versiones mediante Git y repositorio en GitHub.
- Documentación básica de instalación y ejecución del sistema.
- Uso de docker-compose para levantar el proyecto
- El administrador podrá configurar desde el panel administrativo el ID del grupo o chat de Telegram utilizado por el sistema para el envío de mensajes.

Opcional

- Implementación de al menos 3 categorías para organizar las preguntas frecuentes.
- Registro de todas las consultas realizadas por los usuarios, almacenando como mínimo:
 - Fecha y hora
 - Usuario de Telegram
 - Consulta realizada
 - Respuesta proporcionada
- Estadísticas de uso del bot (consultas más frecuentes, cantidad de usuarios, categorías más consultadas, etc.).

Restricciones

- El backend deberá desarrollarse exclusivamente utilizando Python.
- La comunicación entre el bot de Telegram, el panel administrativo y la base de datos deberá realizarse a través de una API REST.
- No se permite almacenar preguntas y respuestas de forma estática dentro del código fuente; toda la información deberá gestionarse desde la base de datos.
- El sistema deberá utilizar una base de datos para almacenar preguntas, respuestas, categorías, usuarios administradores y registros de consultas.
- El acceso al panel administrativo deberá estar protegido mediante autenticación.
- El proyecto deberá ejecutarse utilizando Docker Compose.
- No se permite utilizar plataformas de chatbot de terceros (Chatfuel, ManyChat, Botpress Cloud, Dialogflow, entre otros) para implementar la lógica principal del sistema.
- El bot deberá obtener las respuestas desde la API y la base de datos, evitando respuestas codificadas directamente en el código.

3.3 Requerimientos técnicos

Para el desarrollo de esta práctica se deberán utilizar las siguientes tecnologías:

Lenguajes y herramientas

- Python 3.x para el desarrollo del backend y la API REST.
- Telegram Bot API para la implementación del bot.
- Framework web de libre elección (FastAPI, Flask o equivalente).
- Base de datos de libre elección (MySQL, PostgreSQL, SQLite, MongoDB o equivalente).
- Docker y Docker Compose para la ejecución de la solución.
- Git para el control de versiones.
- GitHub para el almacenamiento del repositorio.

Requisitos mínimos del sistema

- Bot de Telegram completamente funcional.
- API REST operativa para la gestión de información.
- Base de datos configurada y conectada al sistema.
- Panel administrativo con autenticación.
- Usuario administrador preconfigurado:
 - Usuario: **IA1-User**
 - Contraseña: **IA1-password@_new**
- Registro de al menos 20 preguntas frecuentes y sus respectivas respuestas.
- Implementación de al menos 3 categorías para organizar la información.
- Operaciones CRUD completas para preguntas y respuestas.
- Configuración del ID del grupo o chat de Telegram desde el panel administrativo.
- Proyecto ejecutable mediante Docker Compose.
- Documentación de instalación y ejecución.

4. Entregables

Describe los productos concretos que se espera que los estudiantes entreguen al finalizar la práctica. Pueden ser prototipos, informes técnicos, documentación, etc.

Tipo	Descripción
Repositorio del Proyecto	Enlace al repositorio GitHub que contenga el código fuente, documentación, historial de cambios y evidencias del desarrollo.
Backend	Proyecto desarrollado en Python encargado de la lógica de negocio, gestión de datos, autenticación, generación de estadísticas y comunicación con el bot de Telegram.
API REST	Servicio REST funcional que permita la administración de preguntas, respuestas, categorías, configuración del sistema y consultas realizadas por los usuarios.
Bot de Telegram	Bot completamente funcional capaz de recibir consultas, buscar información en la base de datos y responder automáticamente a los usuarios.
Panel Administrativo	Interfaz web que permita administrar preguntas frecuentes, respuestas, categorías, configuración del bot y visualizar estadísticas del sistema.
Base de Datos	Modelo de datos implementado y configurado para el almacenamiento de preguntas, respuestas, categorías, usuarios administradores y registros de consultas.
Diagrama Entidad Relación (ER)	Diagrama que represente las entidades, atributos y relaciones utilizadas dentro de la base de datos del proyecto.
Requerimientos Funcionales	Estas irán en el Manual técnico, describan las funcionalidades implementadas en el sistema.
Requerimientos No Funcionales	Estas irán en el Manual técnico en formato Markdown (.md) que describa aspectos relacionados con rendimiento, seguridad, mantenibilidad, disponibilidad y usabilidad del sistema.
Patrón de arquitectura	Debera de haber un diagrama sobre que patron de arquitectura utilizaron

Manual Técnico	Documento en formato Markdown (.md) que describa la arquitectura implementada, estructura del proyecto, tecnologías utilizadas, modelo de datos, API REST, configuración de Docker Compose y posibles mejoras futuras.
Manual de Usuario	Documento en formato Markdown (.md) que explique la instalación, configuración, ejecución y uso del sistema mediante ejemplos e imágenes.
Evidencias de Funcionamiento	Capturas de pantalla que demuestren el funcionamiento del login, CRUD de preguntas, CRUD de categorías, estadísticas, consultas desde Telegram y configuración del sistema.
Docker Compose	Archivo de configuración necesario para levantar todos los servicios del proyecto mediante Docker Compose.
Historial de Commits	El repositorio deberá contener un mínimo de 5 commits funcionales que evidencien el avance progresivo del desarrollo. No se aceptarán commits masivos realizados el mismo día que representen la totalidad del proyecto.

5. Material de apoyo

Se recomienda consultar los siguientes recursos para comprender el desarrollo de APIs REST, integración con Telegram, bases de datos, Docker y buenas prácticas de desarrollo de software.

Documentación oficial

- FastAPI: <https://fastapi.tiangolo.com>
- Flask: <https://flask.palletsprojects.com>
- Python: <https://docs.python.org/3>
- Telegram Bot API: <https://core.telegram.org/bots/api>
- Python Telegram Bot: <https://python-telegram-bot.org>
- Docker: <https://docs.docker.com>
- Docker Compose: <https://docs.docker.com/compose>
- Git: <https://git-scm.com/doc>
- GitHub: <https://docs.github.com>

6. Recursos y herramientas a utilizar

Software / Hardware

- Computadora personal con sistema operativo Windows, Linux o macOS.
- Python 3.11 o superior.
- Visual Studio Code o cualquier IDE equivalente.
- Docker Desktop o Docker Engine.

- Docker Compose.
- Git para control de versiones.
- Navegador web actualizado.
- Cliente para pruebas de APIs (Postman, Insomnia o equivalente).
- Base de datos de libre elección (PostgreSQL, MySQL, SQLite, MongoDB o equivalente).

Plataformas

- GitHub para alojamiento del repositorio y control de versiones.
- Telegram para la creación y pruebas del bot.
- BotFather para la generación y administración del bot de Telegram.
- Docker Hub (opcional) para almacenamiento de imágenes Docker.
- UEDI para la entrega de la práctica.

Fecha de entrega: 12/06/2026